AUTONOMOUS OBSTACLE AVOIDANCE ROBOT CAR WITH GOOGLE SPEECH-TO-TEXT VOICE CONTROL

PROJECT REPORT

Submitted by

SUBHANKAR DAS– 240301120122

in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

CENTURION UNIVERSITY OF TECHNOLOGY AND MANAGEMENT

ODISHA

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BHUBANESWAR CAMPUS

MAY 2025-26

BONAFIDE CERTIFICATE

Certified that this project report **"AUTONOMOUS OBSTACLE AVOIDANCE ROBOT CAR WITH GOOGLE SPEECH-TO-TEXT VOICE CONTROL"** is the Bonafide work of **"SUBHANKAR DAS"** who carried out the project work under my supervision. This is to further certify to the best of my knowledge, that this project has not been carried out earlier in this institute and the university.

(Dr. Chinmayee Dora)

Assistant Professor (Dept. of ECE)

(Dr. Rajkumar Mohanta)

HEAD OF THE DEPARTMENT / DEAN OF THE SCHOOL

Professor of Computer Science and Engineering

DECLARATION

I hereby declare that the project entitled **"AUTONOMOUS OBSTACLE AVOIDANCE ROBOT CAR WITH GOOGLE SPEECH-TO-TEXT VOICE CONTROL"** submitted for the 4th Semester B. Tech. in Computer Science and Engineering is my original work and the project has not formed the basis for the award of any Degree / Diploma or any other similar titles in any other University / Institute.

Name of the Student: SUBHANKAR DAS

Signature of the Student:

Registration No: 240301120122

Place: Bhubaneswar

Date:

ACKNOWLEDGEMENTS

I wish to express my profound and sincere gratitude to Dr. Chinmayee Dora, Electronics and Communication Engineering, SoET, Bhubaneswar Campus, who guided me into the intricacies of this project nonchalantly with matchless magnanimity.

I thank Dr. Rajkumar Mohanta, Head of the Dept. of Computer Science and Engineering, SoET, Bhubaneswar Campus and Dr. Sujata Chakravarty, Dean, School of Engineering and Technology, Bhubaneswar Campus for extending their support during Course of this investigation.

I would be failing in my duty if I don't acknowledge the cooperation rendered various stages of image interpretation by group members.

I am highly grateful to group members who evinced keen interest and invaluable support in the progress and successful completion of my project work.

I am indebted to my parents for their constant encouragement, co-operation and help. Words of gratitude are not enough to describe the accommodation and fortitude which they have shown throughout my endeavour.

Name of the Student: SUBHANKAR DAS

Signature of the Student:

Registration No: 240301120122

Place: Bhubaneswar

Date:

TABLE OF CONTENTS

CHAPTER / SECTION TITLE PAGE

Bonafide Certificate i
Declaration ii
Acknowledgements iii
Abstract vi
1. Introduction 1
2. Literature Review 2
2.1 Obstacle Avoidance Robotics 2
2.2 ESP32-Based Embedded Robotic Systems 2
2.3 Bluetooth Communication in Robotics 2
2.4 Speech-to-Text Control Systems in Robotics 3
3. System Architecture and Design 3
3.1 Sensing Layer 3
3.2 Processing and Decision Layer 4
3.3 Actuation Layer 4
3.4 Communication Layer — Bluetooth Voice Control Pipeline 4
3.5 Power Management Subsystem 5
4. Methodology 5
4.1 Phase 1: System Design and Component Selection 5
4.2 Phase 2: Hardware Assembly 5
4.3 Phase 3: Power Supply Configuration 6
4.4 Phase 4: Sensor Data Acquisition Logic 6
4.5 Phase 5: Autonomous Navigation Algorithm 6
4.6 Phase 6: Bluetooth Command Reception and Parsing 6
4.7 Phase 7: Voice Mode Safety Logic 7
4.8 Phase 8: Integration Testing and Optimization 7
5. Hardware Components Description 7
5.1 ESP32 Microcontroller 7
5.2 Ultrasonic Sensor (HC-SR04) 8
5.3 IR Proximity Sensor Modules 8
5.4 Motor Driver — TB6612FNG 8
5.5 DC Gear Motors 8
5.6 Servo Motor 9

5.7 Li-Ion Battery Pack and BMS 9
5.8 DC-DC Buck Converter 9

6. Software Implementation 9
6.1 Development Environment and Libraries 10
6.2 System Initialization and Bluetooth Setup 10
6.3 Bluetooth Command Reception 10
6.4 Mode Switching Logic 10
6.5 Directional Command Parsing and Motor Control 11
6.6 Voice Mode Safety Override 11
6.7 Autonomous Navigation Routine 11
6.8 Mobile Application and Google Speech-to-Text Integration 11
7. Results and Discussion 12
7.1 Autonomous Mode Performance 12
7.2 Voice Mode Command Latency and Reliability 12
7.3 Manual Mode Performance 13
7.4 Power System Performance 13
8. Advantages and Limitations 13
8.1 Advantages 13
8.2 Limitations 14
9. Applications 14
10. Future Scope 14
11. Real-Time Implementation 16
12. Conclusion 15
References 16
Appendix A: Key Code Snippets 18
A.1 Command Parsing with Safety Override 18
A.2 Autonomous Navigation Logic 18
A.3 Full Source Code Repository 19
A.5 Pin Configuration Table 19

Abstract

In this paper, present a novel design and practical implementation of an independent multi-mode obstacle avoidance robot using microcontroller ESP32 (ESP-WROOM-32 module), designed and built with voice control feature using mobile application that uses Bluetooth module, and Google Speech-to-Text services. The robot can operate in independent mode, manual control mode and voice control mode, that could easily be switched on during runtime. The independent mode uses combination of single HC-SR04 ultrasonic module for distance measurement in front of robot, and two ultrasonic-based infrared (IR) proximity modules installed on sides of robot body for its lateral detection of obstacle. Distance from left and right sides is scanned by servo-mounted ultrasonic sensor in decision points. Captured information is used for avoiding obstacle and determination of direction for evasion. Voice mode uses mobile application to capture spoken commands of user and transfer them into text format using Google Speech-to-Text services. In text format, these commands are sent via Bluetooth Classic to ESP32 board using BT module. Received information is decoded and processed, and related motor actions are performed using TB6612FNG dual H-bridge motor driver. Important feature in this mode is hardware-based safety function. Whenever ultrasonic sensor captures information within 15 cm, or detected by IR modules installed on sides, robot stops immediately. Powers of robot are provided by two 3.7V Li-ion cells connected in series, and equipped with 20A BMS module (Battery Management System), and DC-DC buck voltage regulator for supply of ESP32 module and motor drivers at required voltages. Experimental results present efficient performance of autonomous multi-mode obstacle avoidance robot that functions at all times in different environments, and proves that there is almost no time delay in voice commands processed using Bluetooth.

Keywords: ESP32, Obstacle Avoidance Robot, Bluetooth Serial, Google Speech-to-Text, TB6612FNG, Autonomous Navigation, Voice Control, IR Sensor, HC-SR04

1. Introduction

Autonomous mobile robotics has seen rapid growth in recent years, driven by the development of embedded computation, miniature sensors and wireless communication. Autonomous robots that can navigate through their environment to gather data, observe objects and people, to collect and deliver packages, to inspect for damage or leaking pipes, to serve as health care aids for the elderly and disabled, and to teach children in futuristic classrooms are becoming more prevalent. A fundamental requirement for these systems is the reliable detection and avoidance of obstacles to achieve safe and continuous robot operation.

Obstacle avoidance approaches can be complex or simple, and can vary from offline path-planning to real-time reaction. Simple threshold-based approaches are a common method for very low-cost implementations, but higher-end microcontrollers enable the use of more sophisticated forms of avoidance. The ESP32 System-on-Chip (SoC) is particularly suited for such tasks as it includes dual core treatment performance backed by internal flash and plenty of RAM, all packed into a very small and affordable module. Its presence of built-in WiFi and Bluetooth enables connectivity.

At the same time cloud-based speech recognition technology became robust enough to be useful for voice control. Services like Google Speech-to--Text recognize speech in real time and output text, which can be sent from your smartphone to your robot using its com-munications embeds. The ESP32 on the ESP WROVER Module supports Bluetooth functionality and thus easily enables this simple communication link. Speech recognition is offloaded to your smartphone while the Bluetooth connection and real-time controls are handled by the robot itself.

In this work, we designed and implemented an autonomous multi-mode robot. The robot could avoid obstacles and be remotely controlled wirelessly. It was able to operate in three modes. The first mode was autonomous mode where it used sensor-based reactive control methodology and servo-assisted scanning to avoid obstacles. The second mode was manual mode where Bluetooth was utilized to receive text commands from a mobile phone. The third mode was voice mode where the robot received speech-to-text commands via mobile application. A safety override was designed to prevent collisions in all control modes.

Our main goals for this project were to achieve real-time obstacle avoidance on low-cost embedded hardware, create a simple voice-controlled system using typical smartphone hardware, and test how the system operated in several different scenarios.

The rest of this paper is organized as follows: Section 2 reviews the related work. Section 3 presents the system architecture of our approach. Section 4 explains our methodology. Section 5 and 6 present our hardware and software implementations respectively. Section 7 discusses the results. We then explain the advantages and disadvantages of the technique in Section 8 and 9 respectively. We also present several applications in Section 10, then Section 11 describes future work. Finally, we conclude in Section 12.

2. Literature Review

2.1 Obstacle Avoidance Robotics

Obstacle avoidance is a fundamental problem in mobile robotics. Although existing methods such as Vector Field Histogram (VFH) based sensor-based obstacle avoidance approach are capable of performing reliable navigation while constructing environmental maps using sensors, they are computationally expensive for embedded microcontrollers. In order to overcome this drawback, lightweight reactive avoidance methods based on threshold logic have been realized on low-cost robots.

Using ultrasonic and infrared (IR) sensors for sensor fusion improves detection accuracy. Ultrasonic sensors are good for longer ranges while infrared sensors provide fast short-range detection. This combination of sensors allows for reliable obstacle avoidance. Furthermore, using servo-based directional scanning further improves detection.

2.2 ESP32-Based Embedded Robotic Systems

The ESP32 microcontroller chosen for this example comes with a dual core processor and includes WiFi and Bluetooth capabilities. It is a suitable chip for embedded robotics and comes at a low cost. Thanks to the strong support of many great libraries you can create a network capable robotic system very quickly. One of these great libraries is the BluetoothSerial library which allows you to setup wireless communication between two rovers in no time. This is particularly easy to integrate into the control system of a rover as you only need to add an extra pin to the existing hardware.

2.3 Bluetooth Communication in Robotics

Bluetooth is a great technology to control robots over a short range, offering both low latency and good stability. Bluetooth on ESP32 does not require an external module, unlike some earlier solutions. Bluetooth Classic SPP (Serial Port Profile) is sufficient to transfer command data from a smart phone / tablet to the robot. Within range, it offers reliable connection compared to WiFi, although the effective range is limited.

2.4 Speech-to-Text Control Systems in Robotics

Most speech-controlled robotic systems use cloud-based speech recognition for higher accuracy, and typically a smartphone is used for speech-to-text, followed by command transmission to the robot via Bluetooth. This approach significantly reduces the workload of the embedded computing resources. Measured response times for received commands are in the range of 0.5 to 1.5 seconds and are practically interactive. For error tolerance and safe operation, recognition results are processed for errors, and, if necessary, certain safety measures are implemented.

3. System Architecture and Design

The system is organized into five principal functional layers: the sensing layer, the processing and decision layer, the actuation layer, the communication layer, and the power management subsystem. Figure 1 illustrates the overall system block diagram.

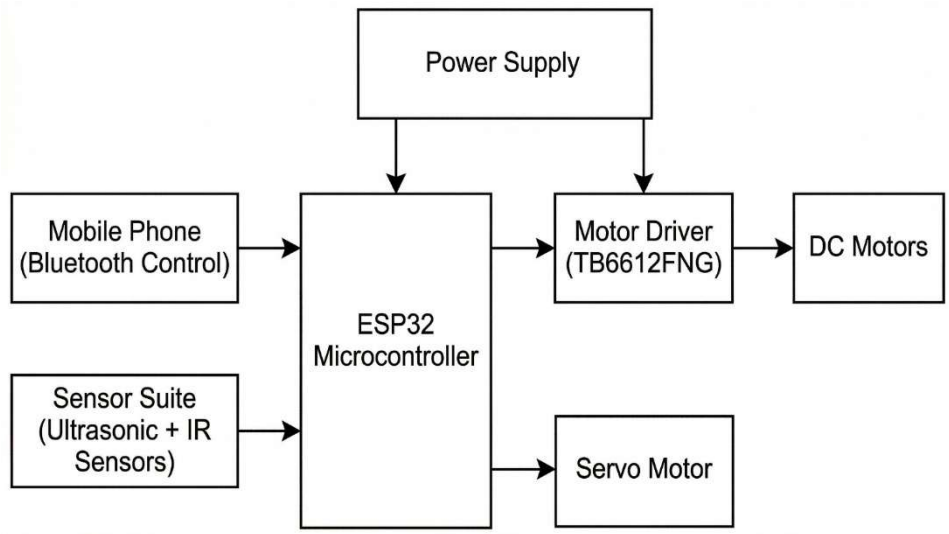


Fig. 1. Overall system architecture showing the five functional layers and their interconnections

3.1 Sensing Layer

The sensing layer consists of an HC-SR04 ultrasonic sensor and two IR proximity sensors. The ultrasonic sensor is mounted on a servo motor, allowing directional scanning. During normal operation, it remains at the forward position (90°) for continuous monitoring. When an obstacle is detected within a threshold distance, the servo rotates toward left (~150°) and right (~30°) to evaluate alternate paths.

The IR sensors are placed at the front-left and front-right edges of the chassis, providing rapid detection of nearby obstacles and edges. The combined use of servo-based ultrasonic scanning and fixed IR sensing improves environmental awareness and enhances obstacle detection reliability.

3.2 Processing and Decision Layer

The ESP32 microcontroller serves as the central processing unit, handling sensor data acquisition, decision-making, communication, and motor control. The system operates in three modes—autonomous, manual, and voice—stored as a runtime variable and updated through Bluetooth commands.

The control logic is implemented using the Arduino framework with ESP32's FreeRTOS support, enabling efficient task handling. The decision-making algorithm varies based on the selected mode and is detailed in Section 4.

3.3 Actuation Layer

The actuation layer includes a TB6612FNG dual H-bridge motor driver and four DC gear motors arranged in a differential drive configuration. The left and right motor pairs are independently controlled, enabling forward motion, turning, and stopping.

Motor speed is regulated using PWM signals generated by the ESP32, while direction is controlled through logic inputs to the driver. The servo motor used for ultrasonic scanning is driven via a dedicated GPIO pin using standard servo control libraries.

3.4 Communication Layer — Bluetooth Voice Control Pipeline

The communication layer enables wireless control through Bluetooth. The system follows a pipeline in which voice commands are captured by a mobile application, converted to text using Google Speech-to-Text, and transmitted to the ESP32 via Bluetooth Serial (SPP).

The ESP32 receives the command string, processes it by trimming and standardizing the input, and executes the corresponding motion command. This approach offloads computationally intensive speech recognition to the smartphone, allowing the embedded system to focus on control execution.

Fig. 2. End-to-end communication flow from user voice command to robot motor actuation via Google Speech-to-Text and Bluetooth SPP.

3.5 Power Management Subsystem

The system is powered by two 3.7V Li-ion batteries connected in series, providing a nominal voltage of 7.4V. A Battery Management System (BMS) ensures protection against overcharging, over-discharging, and short circuits.
A DC-DC buck converter regulates the voltage to 5V for the ESP32 and servo, while sensors operate at 3.3V. The motors are powered directly from the battery to prevent voltage drops and ensure stable operation under varying load conditions.

4. Methodology

4.1 Phase 1: System Design and Component Selection

The system supports autonomous, manual, and voice modes with reliable obstacle detection and wireless control. The ESP32 handles processing and Bluetooth, while the HC-SR04 and IR sensors provide complementary detection. The TB6612FNG motor driver ensures efficient control, and GPIOs were assigned to avoid conflicts.

4.2 Phase 2: Hardware Assembly

All components were mounted on a four-wheel drive chassis. Motors were connected to the motor driver, sensors were positioned for optimal coverage, and the ESP32 was centrally placed. Wiring was completed using jumper connections and verified before power-up.

4.3 Phase 3: Power Supply Configuration

Two Li-ion batteries in series provided 7.4V, managed by a BMS. A buck converter regulated voltage to 5V for the ESP32, while motors were powered directly from the battery. A common ground ensured stable operation.

4.4 Phase 4: Sensor Data Acquisition Logic

The ultrasonic sensor measures distance via pulse timing, while IR sensors detect obstacles digitally. A brief debounce delay minimizes noise and false triggers, improving reliability.

4.5Phase 5: Autonomous Navigation Algorithm

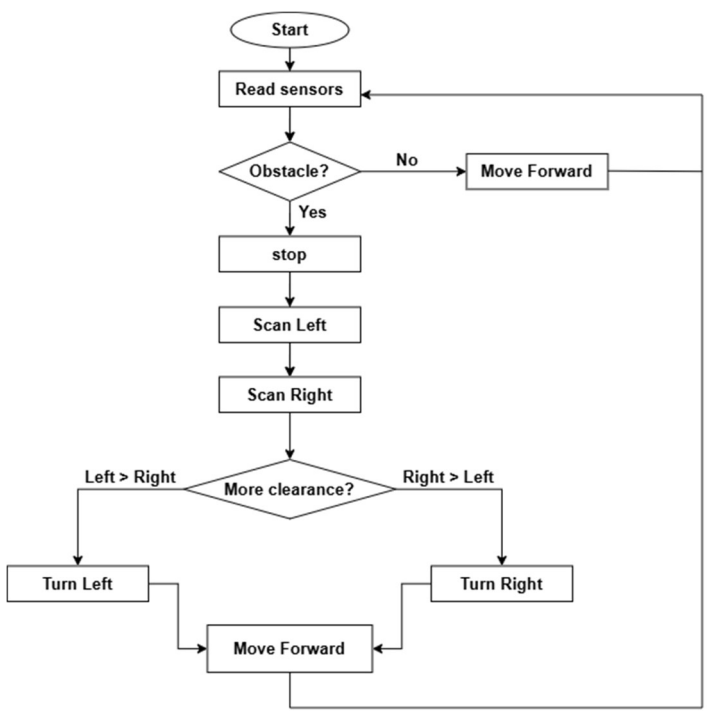


Fig. 3. Decision flowchart for autonomous obstacle avoidance mode, including servo-scan-based directional selection.

4.6 Phase 6: Bluetooth Command Reception and Parsing

The ESP32 continuously checks for incoming Bluetooth data. Commands are read, formatted, and parsed to identify mode changes or movement instructions. Valid commands trigger corresponding motor actions.

4.7 Phase 7: Voice Mode Safety Logic

In voice mode, a safety override ensures collision prevention. After each command, sensors are rechecked, and the robot is stopped if an obstacle is detected within a threshold distance. This prevents accidents caused by delayed or incorrect voice commands. Figure 4 illustrates the voice mode command processing and safety override flow.

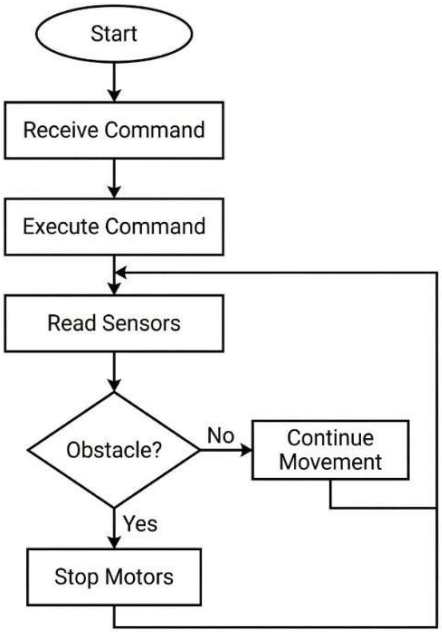


Fig. 4. Flowchart illustrating Bluetooth command reception, parsing, execution, and the post-command safety override in voice mode.

4.8 Phase 8: Integration Testing and Optimization

The system was tested in different indoor environments. Performance metrics included navigation reliability and command response time. Issues such as connection instability were addressed, and movement behavior was refined to reduce oscillations.

5. Hardware Components Description

The robot system integrates eight categories of hardware components, each selected for functional suitability, compatibility with the ESP32, and cost-effectiveness. Figure 5 shows the assembled hardware with component labels.

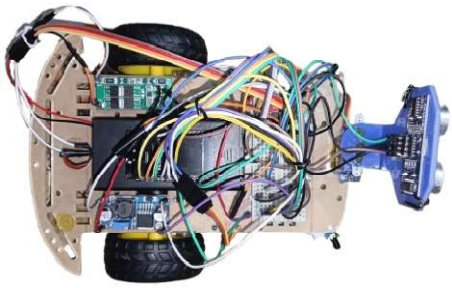


Fig. 5. Photograph of the assembled robot (placeholder) having ESP32, motor driver, sensors, battery pack, and servo assembly.

5.1 ESP32 Microcontroller

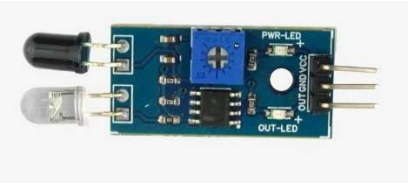
The ESP32 DevKit V1 serves as the central controller, handling sensing, processing, communication, and actuation. It provides multiple GPIO pins supporting PWM, ADC, and serial communication. Its integrated Bluetooth enables wireless control, while PWM channels are used for motor speed and servo control.

5.2 Ultrasonic Sensor (HC-SR04)

The HC-SR04 measures distance using ultrasonic waves and is used for forward obstacle detection. It operates within a range of a few centimeters to several meters and provides reliable distance estimation for navigation. A voltage divider ensures safe interfacing with the ESP32.

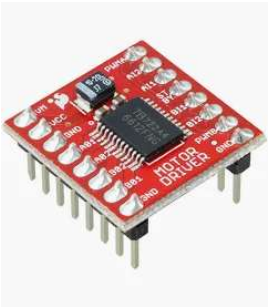
5.3 IR Proximity Sensor Modules

Two IR sensors are used for short-range obstacle detection on the left and right sides. They provide digital outputs based on reflected infrared signals and help detect nearby objects and edges. Their fast response complements the ultrasonic sensor.



5.4 Motor Driver — TB6612FNG

The TB6612FNG motor driver controls the speed and direction of the motors. It uses PWM for speed control and logic inputs for direction. Compared to traditional drivers, it offers better efficiency and lower heat generation, making it suitable for compact robotic systems.



5.5 DC Gear Motors

Two DC gear motors drive the robot in a differential configuration. The gear reduction provides sufficient torque and controlled speed for stable movement. Independent control of left and right motors enables turning and maneuvering.

5.6 Servo Motor

A micro servo motor is used to rotate the ultrasonic sensor for directional scanning. It allows the robot to evaluate multiple directions before making navigation decisions, improving obstacle avoidance performance.

5.7 Li-Ion Battery Pack and BMS

The system is powered by two Li-ion cells connected in series. A Battery Management System (BMS) ensures safe operation by protecting against overcharge, over-discharge, and short circuits.

5.8 DC-DC Buck Converter

A DC-DC buck converter regulates the battery voltage to a stable 5V supply for the ESP32 and other components. This ensures consistent performance despite battery voltage variations.

6. Software Implementation

The firmware is implemented in C++ using the Arduino IDE with the Espressif ESP32 Arduino core. The software architecture follows a single-file sketch pattern with well-defined functional sections covering Bluetooth initialization, sensor reading utilities, motor control primitives, the autonomous navigation routine, and the command dispatcher. The following subsections describe each component in detail, with reference to the code excerpts provided in the Appendix.

6.1 Development Environment and Libraries

The system is implemented using the Arduino IDE with ESP32 support. Key libraries include BluetoothSerial for wireless communication and ESP32Servo for servo control. Standard Arduino functions are used for GPIO, timing, and sensor interaction.

6.2 System Initialization and Bluetooth Setup

During setup, all GPIO pins are configured, the motor driver is enabled, and the servo is initialized to its default position. Bluetooth is activated with a device name, allowing pairing with a mobile application. Serial communication is also initialized for debugging.

6.3 Bluetooth Command Reception

The system continuously checks for incoming Bluetooth data. Commands are read as complete strings, cleaned, and converted to lowercase for consistent processing. The processed input is passed to a command handler function.

6.4 Mode Switching Logic

The command handler identifies mode-switching commands (autonomous, manual, voice) and updates the system mode accordingly. This ensures real-time switching without unintended motor actions.

6.5 Directional Command Parsing and Motor Control

Directional commands such as forward, backward, left, right, and stop are mapped to motor control functions. PWM signals regulate speed, while logic inputs control direction. Differential drive enables flexible movement and turning.

6.6 Voice Mode Safety Override

In voice mode, a safety mechanism checks sensor data after every command. If an obstacle is detected within a threshold distance, the robot stops immediately, preventing collisions due to delayed or incorrect commands.

6.7 Autonomous Navigation Routine

In autonomous mode, the robot moves forward when the path is clear. If an obstacle is detected, it stops and performs servo-based scanning to measure left and right distances. The robot then turns toward the direction with greater clearance before resuming movement.

6.8 Mobile Application and Google Speech-to-Text Integration

A mobile application is used for voice input and command transmission. Speech is converted to text using Google Speech-to-Text and sent via Bluetooth to the ESP32. This approach reduces processing load on the microcontroller while maintaining responsive control.

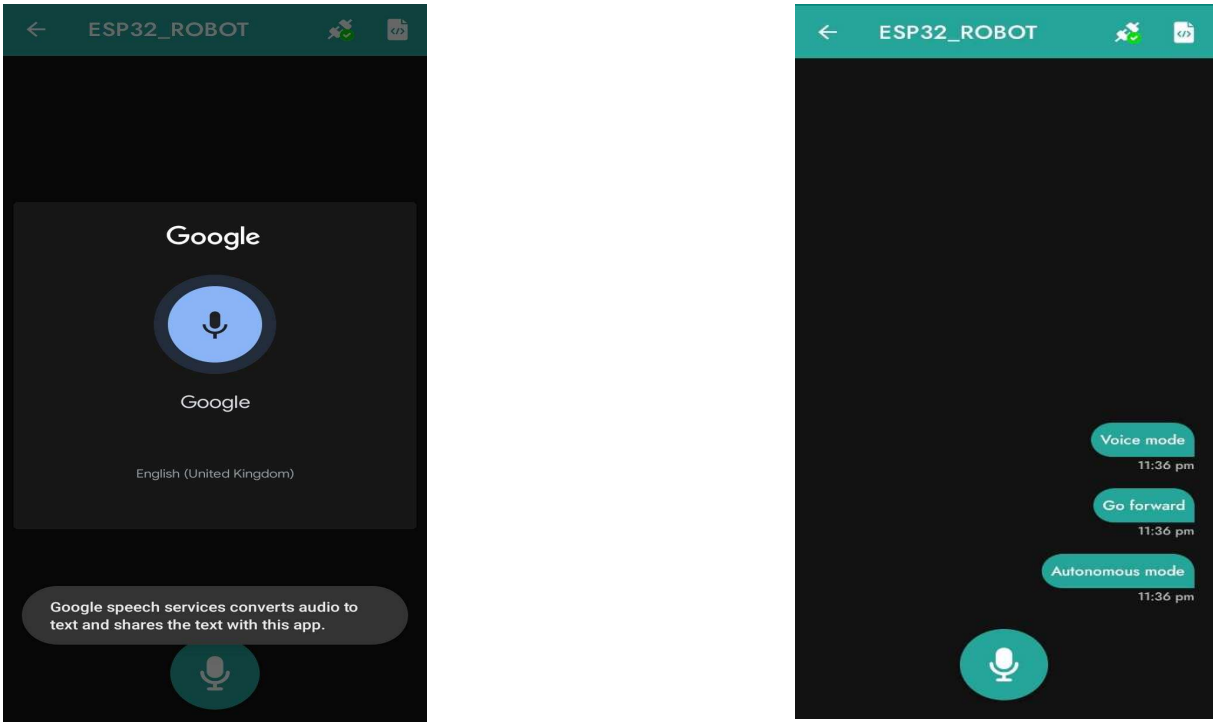


Fig. 6. Screenshot placeholder of the mobile application showing speech-to-text input, command confirmation

7. Results and Discussion

7.1 Autonomous Mode Performance

The robot was tested in three indoor environments: a corridor, an open room with obstacles, and a cluttered workspace. In the corridor and open-room scenarios, the robot successfully completed all trials without collisions. The servo-based scanning mechanism improved navigation by selecting optimal turning directions, reducing secondary collisions compared to fixed-turn methods.

In the cluttered environment, one collision occurred due to an obstacle below the ultrasonic sensor's detection range, highlighting a limitation of the sensing configuration. The IR sensors effectively detected lateral obstacles and supported reliable avoidance behavior.

7.2 Voice Mode Command Latency and Reliability

Voice commands were tested under typical indoor conditions. The system showed an average response latency of approximately 0.8 to 1.8 seconds, primarily due to speech recognition processing. Bluetooth transmission and ESP32 execution introduced minimal delay.

Command accuracy was high for simple commands, with occasional errors under noisy conditions. The safety override mechanism consistently prevented collisions by stopping the robot when obstacles were detected within the threshold distance.

7.3 Manual Mode Performance

In manual mode, the robot responded to Bluetooth text commands with latency below 150 ms. All commands were executed correctly, demonstrating reliable communication and robust command parsing.

7.4 Power System Performance

The power system maintained stable operation throughout testing. No voltage drops or protection triggers were observed under load. The system achieved an estimated runtime of 90–120 minutes, sufficient for continuous operation and demonstrations.

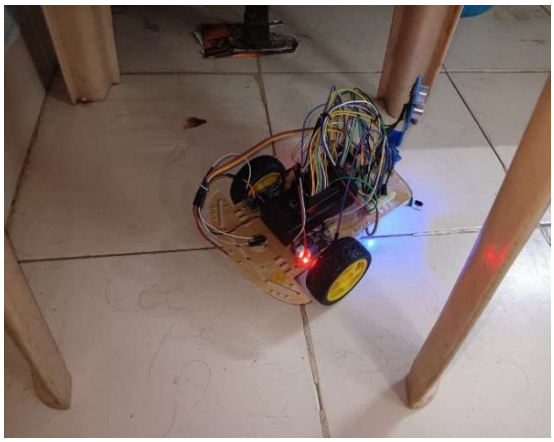


Fig. 7. Photograph placeholder of the robot navigating the obstacle course during autonomous mode evaluation.

8. Advantages and Limitations

8.1 Advantages

The system offers a cost-effective and efficient design by utilizing the ESP32's built-in Bluetooth, eliminating the need for external communication modules. Bluetooth SPP provides a reliable, low-latency control link without requiring continuous internet connectivity.

The use of smartphone-based speech recognition ensures high accuracy without burdening the microcontroller with heavy processing tasks. The safety override mechanism enhances reliability by preventing collisions during voice operation.

Additionally, the multi-mode architecture (autonomous, manual, and voice) provides flexibility for different use cases, while the servo-based ultrasonic scanning improves navigation efficiency compared to fixed-direction approaches.

8.2 Limitations

The system has limited obstacle coverage due to its forward-facing sensor configuration, making it less effective in detecting low-height or wide-angle obstacles.

Voice control depends on cloud-based speech recognition, introducing latency and requiring internet connectivity. The reactive navigation algorithm lacks environmental mapping, which can lead to inefficient movement or oscillations in complex environments.

Finally, the limited range of Bluetooth communication restricts operation to short distances.

9. Applications

The proposed system is applicable across multiple domains due to its flexible multi-mode design.
In educational settings, it serves as an effective platform for learning embedded systems, robotics, sensor integration, and wireless communication. The multi-mode operation allows students to understand and experiment with different control approaches within a single system.
In domestic environments, the robot can be adapted for simple tasks such as object delivery or voice-controlled assistance. The safety override mechanism makes it suitable for use around people, pets, and fragile objects.
For accessibility applications, voice control enables users with limited mobility to operate the robot without physical interaction, improving usability and independence.
In research and prototyping, the system provides a modular and extensible platform for developing advanced navigation algorithms, sensor fusion techniques, and enhanced control systems. New functionalities can be added without major modifications to the existing architecture.
In light industrial scenarios, the robot can support supervised operation, where autonomous behavior can be temporarily overridden through manual or voice commands, similar to real-world robotic control systems.

10. Future Scope

Several improvements can enhance the system's performance and capability.

In terms of sensing, additional ultrasonic sensors at angled positions and a downward-facing IR sensor can reduce blind spots and improve obstacle coverage. The current servo-based scanning can be extended to continuous 360° scanning, enabling basic environment mapping.

From an algorithmic perspective, replacing the reactive navigation approach with map-based planning (such as grid-based pathfinding) would improve efficiency and eliminate oscillation issues. Advanced techniques like lightweight SLAM could further enable navigation in previously explored environments.

The voice control system can be improved by implementing on-device speech recognition, reducing latency and removing cloud dependency. Expanding the command set to include speed control and task-specific actions would enhance usability. Integration with IoT platforms using protocols like MQTT can enable smart home interaction.

Hardware enhancements such as omnidirectional wheels can improve maneuverability, while adding a camera module can enable real-time video feedback and support future vision-based features.

11. Real-Time Implementation

This section documents the physical realization of the system through annotated image placeholders corresponding to key stages of implementation and testing. These images should be replaced with actual photographs in the final submission.

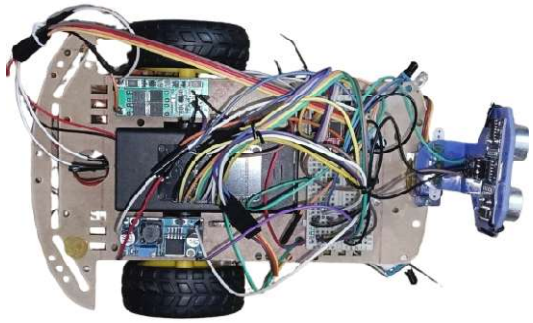


Fig. 8. Top-view photograph of the fully assembled robot showing ESP32 board, motor driver, sensor wiring, and battery compartment (placeholder).

Figure 8 illustrates the component layout on the chassis. The ESP32 DevKit is mounted centrally, with the TB6612FNG motor driver positioned adjacent to it for short wire runs to the motor terminals. The battery holder and BMS are mounted at the rear of the chassis, and the buck converter is positioned between the battery compartment and the ESP32 power input.

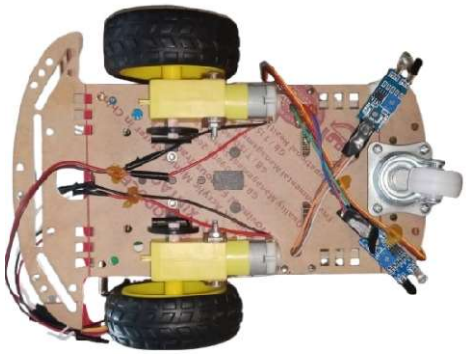


Fig. 9. Underside view showing motor wiring and wheel assembly configuration (placeholder).

Figure 9 shows the motor wiring from each of the four DC gear motor units to the TB6612FNG output channels. The wiring is routed along the chassis frame and secured with cable ties to prevent interference with the wheel rotation.

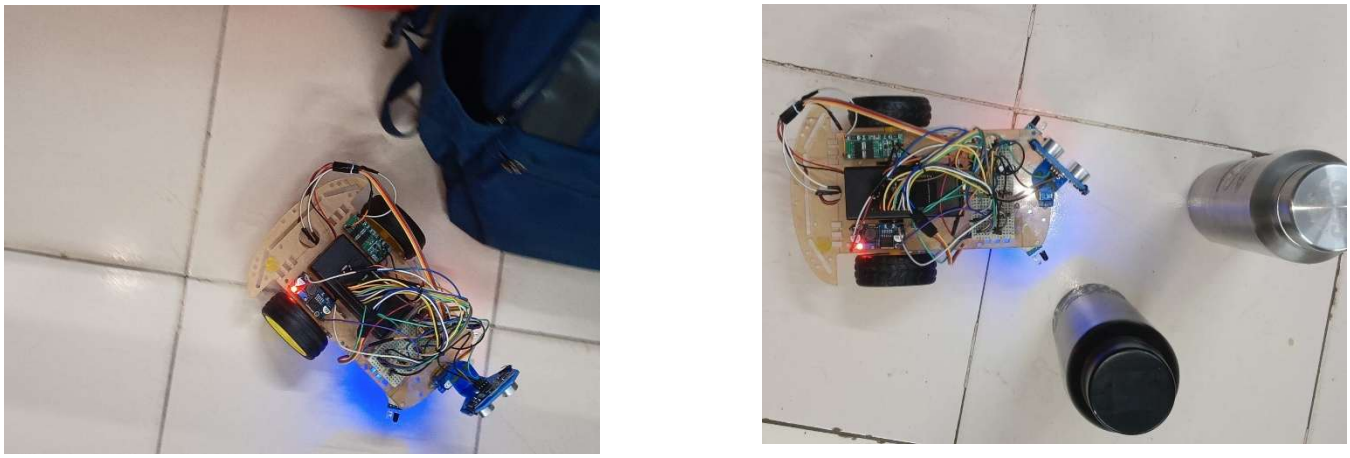


Fig. 10. Photograph of the robot navigating an obstacle course during autonomous mode evaluation (placeholder).

Figure 10 documents the autonomous obstacle avoidance test conducted in an open-room environment with six cardboard box obstacles distributed at varying positions. The robot's traversal path is marked with coloured tape to illustrate the navigation trajectory.

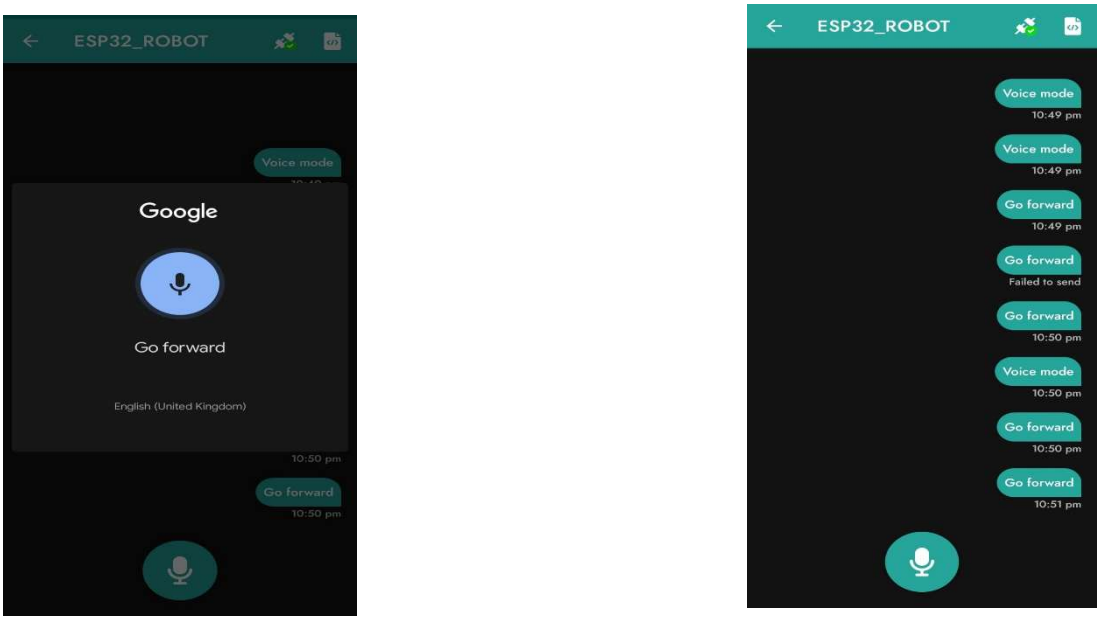


Fig. 11. Photograph of the user issuing a voice command via the mobile application while the robot responds (placeholder).

Figure 11 illustrates the voice control interaction. The user's smartphone running the Bluetooth voice control application is shown paired with the ESP32 and displaying the recognized text command prior to transmission.

12. Conclusion

This paper has presented the design, implementation, and evaluation of a multi-mode autonomous obstacle avoidance robot based on the ESP32 microcontroller, integrating Bluetooth-based voice control via a mobile application employing Google Speech-to-Text. The system successfully demonstrates three independently functional and dynamically switchable operational modes: an autonomous mode employing HC-SR04 ultrasonic sensing with servo-assisted directional scanning and IR lateral detection; a manual text command mode over Bluetooth SPP; and a voice mode in which speech-to-text-converted commands are transmitted from a smartphone to the ESP32 via Bluetooth, subject to a post-command hardware safety override.

Experimental evaluation confirmed reliable autonomous navigation in structured indoor environments, with the servo-scan-assisted decision algorithm demonstrably reducing secondary collision events compared to a fixed-turn baseline. Voice mode end-to-end command latency of 0.8 to 1.8 seconds was observed to be acceptable for low-speed robotic control applications. The safety override functioned correctly in all tested scenarios, halting the robot when sensor-detected obstacles were within the 15 cm safety threshold regardless of the issued command. The Bluetooth SPP communication link demonstrated sub-150 ms latency in manual mode, confirming that the voice pipeline latency is attributable to the speech recognition process rather than the wireless link.

The system demonstrates that the ESP32 is a capable and sufficient platform for integrating multi-modal robotic control — combining reactive sensor-based navigation, wireless communication, and voice command interpretation — within a cost-effective, battery-powered mobile package. The work provides a documented, modifiable foundation for further research in voice-controlled robotics, safety-critical embedded systems, and multi-mode human-robot interaction.

References

[1] Espressif Systems, "ESP32 Technical Reference Manual," Version 5.1, Espressif Systems, Shanghai, China, 2023. [Online].
Available:https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf

[2] J. Borenstein and Y. Koren, "The vector field histogram — fast obstacle avoidance for mobile robots," IEEE Transactions on Robotics and Automation, vol. 7, no. 3, pp. 278–288, Jun. 1991.

[3] A. Elfes, "Using occupancy grids for mobile robot perception and navigation," IEEE Computer, vol. 22, no. 6, pp. 46–57, Jun. 1989.

[4] S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics. Cambridge, MA, USA: MIT Press, 2005.

[5] Toshiba Semiconductor, "TB6612FNG Motor Driver IC Datasheet," Toshiba Corporation, Tokyo, Japan, 2015. [Online].
Available: https://www.sparkfun.com/datasheets/Robotics/TB6612FNG.pdf

[6] M. Maier, A. Budinsky, and P. Herber, "Bluetooth Low Energy in embedded robotics: A performance evaluation," in Proc. IEEE International Conference on Emerging Technologies and Factory Automation (ETFA), pp. 1–6, Sep. 2019.

[7] Google LLC, "Cloud Speech-to-Text API Documentation," Google Cloud, Mountain View, CA, USA, 2024. [Online].
Available: https://cloud.google.com/speech-to-text/docs

[8] P. Sermanet, C. Frome, and E. Real, "Attention for fine-grained categorization," arXiv preprint arXiv:1412.7054, 2014. [Online].
Available: https://arxiv.org/abs/1412.7054

[9] R. Kumar and A. Singh, "Design of autonomous obstacle-avoiding robot using Arduino and ultrasonic sensor," in Proc. International Conference on Communication and Electronics Systems (ICCES), IEEE, 2018, pp. 914–918.

[10] K. Cho, D. Kim, and H. Bang, "Voice command controlled mobile robot using Android speech recognition and Bluetooth communication," in Proc. IEEE International Conference on Consumer Electronics (ICCE), IEEE, 2017, pp. 73–74.

[11] V. Kaur and R. Malhotra, "Speech-to-text conversion techniques: A comparative review," International Journal of Computer Applications, vol. 164, no. 1, pp. 21–25, Apr. 2017.

[12] Renesas Electronics, "HC-SR04 Ultrasonic Ranging Module Datasheet," Renesas, 2022. [Online]. Available: https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf

Appendix A. Key Code Snippets

A.1 Command Parsing with Safety Override

```
void handleCommand(String cmd)

if (cmd == "mode auto") { mode = "auto"; return; }
if (cmd == "mode manual") { mode = "manual"; return; }
if (cmd == "mode voice") { mode = "voice"; return; }
if (mode == "manual" || mode == "voice") {
if (cmd == "go forward") moveForward();
else if (cmd == "go back") moveBackward();
 else if (cmd == "turn left") turnLeft();
 else if (cmd == "turn right") turnRight();
else if (cmd == "stop") stopMotors();
// Safety override in voice mode
if (mode == "voice") {
long dist = getDistance();
 bool irL = digitalRead(IR_LEFT) == LOW;
bool irR = digitalRead(IR_RIGHT) == LOW;
 if (dist < 15 || irL || irR) stopMotors();
}
}
}
```

A.2 Autonomous Navigation Logic

The command reception logic within the main loop is identical to the BluetoothSerial polling section shown in Appendix A.1. The handleCommand() function is called with the cleaned command string.

```
void autonomousNav() {
 long dist = getDistance();
 bool irL = digitalRead(IR_LEFT) == LOW;
 bool irR = digitalRead(IR_RIGHT) == LOW;

 if (dist > 30 && !irL && !irR) {
 moveForward();
} else {
stopMotors();
delay(300);
int dLeft = scanAngle(150);
 int dRight = scanAngle(30);
if (dLeft > dRight) turnLeft();
 else turnRight();
delay(400); } }
```

A.3 Full Source Code Repository

Source Code and Project Repository

The complete project source code, circuit schematics, wiring diagrams, and supporting documentation are available in the public GitHub repository:

Repository URL:
https://github.com/Subh-DS/Robo-Car

Repository Structure:

- **/firmware** — ESP32 Arduino code with all operational modes and Bluetooth integration
- **/schematics** — Circuit schematic and wiring diagrams
- **/mobile_app** — Source code for the voice control application
- **/docs** — Bill of materials, pin configuration, and assembly details
- **/media** — Images of hardware setup and testing

The repository enables full system reproduction and further development.

A.5 Pin Configuration Table

ESP32 GPIO	Connected Component	Signal Type	Function
GPIO 5	HC-SR04 TRIG	Digital OUT	Ultrasonic trigger pulse
GPIO 18	HC-SR04 ECHO	Digital IN	Echo duration measurement
GPIO 34	IR Sensor Left	Digital IN	Left obstacle detection
GPIO 35	IR Sensor Right	Digital IN	Right obstacle detection
GPIO 13	Servo Motor Signal	PWM OUT	Ultrasonic sweep scanning
GPIO 25	TB6612FNG AIN1	Digital OUT	Motor A direction pin 1
GPIO 26	TB6612FNG AIN2	Digital OUT	Motor A direction pin 2
GPIO 27	TB6612FNG PWMA	PWM OUT	Motor A speed control
GPIO 14	TB6612FNG BIN1	Digital OUT	Motor B direction pin 1
GPIO 12	TB6612FNG BIN2	Digital OUT	Motor B direction pin 2
GPIO 33	TB6612FNG PWMB	PWM OUT	Motor B speed control
GPIO 32	TB6612FNG STBY	Digital OUT	Motor driver standby
3.3V / GND	Sensors, ESP32 VCC/GND	Power	Logic-level power supply
VIN (5V)	Buck Converter Output	Power	Board power input